
A Minimalist Approach to Offline Reinforcement Learning

Scott Fujimoto^{1,2} Shixiang Shane Gu²

¹Mila, McGill University

²Google Research, Brain Team

scott.fujimoto@mail.mcgill.ca

Abstract

*警告：该PDF由GPT-Academic开源项目调用大语言模型+Latex翻译插件一键生成，版权归原文作者所有。翻译内容可靠性无保障，请仔细鉴别并以原文为准。项目Github地址 https://github.com/binary-husky/gpt_academic/。当前大语言模型: gpt-4o, 当前语言模型温度设定: 1。为了防止大语言模型的意外谬误产生扩散影响，禁止移除或修改此警告。

离线强化学习（RL）定义了从固定数据批次中学习的任务。由于来自分布外动作的价值估计错误，大多数离线RL算法采取限制或正则化策略以匹配数据集中包含的动作的方法。基于现有的RL算法，将RL算法修改为离线运行会带来额外的复杂性。离线RL算法引入新的超参数，并经常利用生成模型等二级组件，同时调整基础的RL算法。在本文中，我们旨在通过最少的改动使深度RL算法工作。我们发现，只需在在线RL算法的策略更新中添加行为克隆项并规范化数据，就能达到最先进的离线RL算法的性能。由此产生的算法实现和调参简单，并且通过去除之前方法的额外计算开销，将总体运行时间减少了超过一半。

1 Introduction

传统上，强化学习（RL）被视为一种在线学习的范式，其中代理与其环境之间的互动对于代理如何学习至关重要。在离线强化学习（历史上称为批处理强化学习）中，代理从一个固定大小的数据集学习，这些数据集是由某些任意的、可能是未知的过程收集的[Lange et al., 2012]。消除与环境互动的需求是值得注意的，因为数据收集通常可能既昂贵又具有风险，或者在现实世界的应用中存在其他挑战。因此，离线强化学习使得能够使用先前记录的数据或利用专家（如人类操作员），而无需承担未训练强化学习代理带来的任何风险。

遗憾的是，离线强化学习的主要好处——缺乏环境互动，也是使其成为一项具有挑战性任务的原因。虽然大多数离策强化学习算法适用于离线设置，但由于“外推误差”：即政策评估中的错误，代理倾向于对其状态-动作对的值进行不良估计，未包含在数据集中的状态-动作对的值，这反过来影响了政策改进，代理学会了偏好其价值被高估的分布外的动作，导致性能不佳[Fujimoto et al., 2019b]。解决这个问题的方案围绕着这样一个想法，即学得的政策应保持接近数据生成过程（或行为策略），根据这种“接近度”的实施方式，这一类方法

被赋予了各种名称（例如：批处理约束[Fujimoto et al., 2019b]，KL控制[Jaques et al., 2019]，行为正则化[Wu et al., 2019]，或策略约束[Levine et al., 2020]）。

尽管提出了许多离线强化学习的方法，我们注意到很少有真正“简单”的方法，即使那些声称只需对底层在线强化学习算法进行少量添加就能工作的算法，实际上也进行了大量的实现级别的调整。在其他情况下，存在未提及的超参数或次级组件，如生成模型，这使得离线强化学习算法难以复制，甚至更难调整。此外，这些细节的混合减慢了算法的运行时间，并使得性能增益的因果归因及技术在不同算法间的转移变得困难，正如许多在线强化学习算法的情况一样[Henderson et al., 2017, Tucker et al., 2018, Engstrom et al., 2020, Andrychowicz et al., 2021, Furuta et al., 2021]。这激发了对更简约的离线强化学习方法的需求。

在本文中，我们提出：我们能否通过最少的更改使深度强化学习算法离线工作？我们发现，我们可以通过对TD3算法[Fujimoto et al., 2018]的策略更新步骤进行单一调整来匹配最先进离线强化学习算法的性能。TD3的策略 π 使用确定性策略梯度[Silver et al., 2014]进行更新。

$$\pi = \operatorname{argmax}_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} [Q(s, \pi(s))]. \quad (1)$$

我们提出的修改，TD3+BC，是简单地添加一个行为克隆项来正则化策略：

$$\pi = \operatorname{argmax}_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\lambda Q(s, \pi(s)) - (\pi(s) - a)^2 \right], \quad (2)$$

通过一个单一的超参数 λ 来控制正则项的强度。这一修改只需调整一行代码即可完成。此外，我们注意到，通过对数据集中的状态进行归一化处理，使其均值为 0 且标准差为 1，可以提高所学策略的稳定性。重要的是，这些是我们对底层深度强化学习算法所做的唯一修改。为了促进可重复性，我们的所有代码都是开源的¹。

我们在 D4RL 基准测试的连续控制任务上评估了我们对 TD3 算法的最小修改 [Fu et al., 2020]。我们发现，我们的算法在与许多离线强化学习算法的比较中表现良好，同时实现起来要容易得多，并且将所需的计算成本减少了超过一半。我们的极简方法的有效性令人惊讶，这表明在离线强化学习的背景下，更简单的方法可能因为过于简单而被忽视，取而代之的是更为复杂的算法贡献。

2 Related Work

虽然据我们所知，我们是第一个将TD3与行为克隆（BC）结合用于离线强化学习（RL）的研究团队，但值得注意的是，将RL与BC以及其他模仿学习方法结合起来的想法之前已被许多作者探讨过。

RL + BC。为了通过例子加速强化学习（即从演示中学习 [Atkeson and Schaal, 1997]），行为克隆已作为策略优化的一种正则化手段与DDPG [Lillicrap et al., 2015, Nair et al., 2018, Goecks et al., 2020] 和自然策略梯度 [Kakade, 2001, Rajeswaran et al., 2017] 结合使用，但采用了修改回放缓冲区和预训练等额外的复杂技术。与我们的工作最相似的是SAC+BC基线 [Haarnoja et al., 2018] 来自 Nair et al. [2020] 和一个未发表的课程项目 [Booher]，该项目将PPO [Schulman et al., 2017] 与BC结合。

RL + 模仿学习。除了直接将行为克隆与RL结合外，模仿学习还以多种方式与强化学习相结合，例如与对抗方法混合 [Zhu et al., 2018, Kang et al., 2018]、用于预训练 [Pfeiffer et al., 2018]、修改回放缓冲区 [Večerík et al., 2017, Gulcehre et al., 2020]、调整价值函数 [Kim et al., 2013, Hester et al., 2017] 或者奖励塑形 [Judah et al., 2014, Wu et al., 2021]。在所有情况下，这些方法都使用演示来克服探索挑战或提高RL代理的学习速度。

¹https://github.com/sfujim/TD3_BC

离线RL。如前所述，离线RL方法通常依赖于某种“靠近”数据的方法。这可以通过估计行为策略并定义显式策略参数化来实现 [Fujimoto et al., 2019b, Ghasemipour et al., 2020]，或者通过使用发散正则化 [Jaques et al., 2019, Kumar et al., 2019, Wu et al., 2019, Siegel et al., 2020, Guo et al., 2021, Kostrikov et al., 2021]。其他方法使用加权版本的BC来倾向于高优势动作 [Wang et al., 2018, Peng et al., 2019, Siegel et al., 2020, Wang et al., 2020, Nair et al., 2020] 或对数据的一个显式子集执行BC [Chen et al., 2020]。一些方法根据计数调整了有效动作集 [Laroche et al., 2019] 或学习的行为策略 [Fujimoto et al., 2019a]。另一个方向是在价值估计中实现一种悲观主义形式的发散正则化 [Nachum et al., 2019, Kumar et al., 2020, Buckman et al., 2020]。

RL算法的元分析。关于在线RL算法的元分析研究相当丰富。有些研究集中在实验协议的不足上 [Henderson et al., 2017, Osband et al., 2019]，而另一些研究则探讨了算法中细微实现细节的作用 [Tucker et al., 2018, Engstrom et al., 2020, Andrychowicz et al., 2021, Furuta et al., 2021]。例如，Tucker et al. [2018], Engstrom et al. [2020] 发现某些算法的卓越性能更多地取决于或甚至是偶然由于细微的实现而非算法差异。Furuta et al. [2021] 研究了两大类离策略算法，大多数离线算法基于这些算法，并发现少量微妙的实现细节与特定算法高度协同且至关重要，使得性能增益归因变得困难。最近的离线研究也呈现类似趋势，需要大量的实现修改才能达到高效的算法性能（见Table 1）。相比之下，我们的算法通过只添加几行代码就改进了现有的TD3。我们的结果表明，除了强调算法创新和复杂性外，社区还可以

3 Background

RL。强化学习（RL）是一个旨在处理顺序任务的框架。通常，该问题由一个马尔可夫决策过程（MDP） $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma)$ 定义，其中状态空间为 $\mathcal{S}$ ，动作空间为 $\mathcal{A}$ ，标量奖励函数为 $\mathcal{R}$ ，转移动力学为 p ，折扣因子为 γ [Sutton and Barto, 1998]。RL 代理的行为由策略 π 确定，该策略将状态映射到动作（确定性策略），或将状态映射到动作的概率分布（随机策略）。RL 代理的目标是最大化预期折现收益 $\mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_{t+1}]$ ，这是一旦在 MDP 中遵循该策略时，预期累积奖励的总和，其中未来奖励的重要性由折扣因子决定。我们通过价值函数来衡量这一目标，该函数衡量在状态 s 下采取行动 a 后的预期折现收益： $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_{t+1} | s_0 = s, a_0 = a]$ 。

BC。训练策略的另一种方法是模仿专家或行为策略。行为克隆（BC）是一种模仿学习的方法 [Pomerleau, 1991]，其中策略通过监督学习直接模仿所提供数据集中的动作。与 RL 不同，这种方法高度依赖于数据收集过程的性能。

离线 RL。离线 RL 打破了代理能够与环境交互的假设。相反，代理被提供了一个固定的数据集，这些数据集是由某些未知的数据生成过程（如一系列行为策略）收集的。由于代理失去了根据当前信念探索 MDP 的机会，而只能从提供的数据中推断出良好的行为，因此这种设置可能更具挑战性。

离线 RL 面临的一个挑战是外推误差问题 [Fujimoto et al., 2019b]，即由于选择了不在数据集中包含的动作而在近似价值函数中引起的泛化误差。简单来说，评估与行为策略有显著差异的策略的预期价值是困难的。因此，算法采用了限制或正则化策略，使其保持接近数据集中的动作 [Levine et al., 2020]。

	CQL [Kumar et al., 2020]	Fisher-BRC [Kostrikov et al., 2021]	TD3+BC (Ours)
Algorithmic Adjustments	Add regularizer to critic [†] Approximate logsumexp with sampling [‡]	Train a generative model ^{†‡} Replace critic with offset function Gradient penalty on offset function [†]	Add a BC term [†]
Implementation Adjustments	Architecture ^{†‡} Actor learning rate [†] Pre-training actor Remove SAC entropy term Max over sampled actions [‡]	Architecture ^{†‡} Reward bonus [†] Remove SAC entropy term	Normalize states

表 1: 实现对底层基础强化学习算法的改变。[†] 表示添加额外超参数的细节, 而 [‡] 表示增加计算成本的部分。

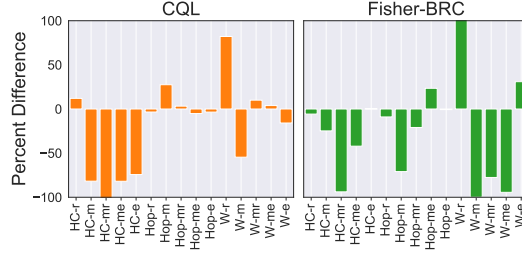


图 1: 离线强化学习算法及其简化版本（移除了对其底层算法的实现调整）的性能百分比差异。HC = HalfCheetah, Hop = Hopper, W = Walker, r = 随机, m = 中等, mr = 中等-回放, me = 中等-专家, e = 专家。巨大的性能下降表明, 实现的复杂性对于这些先前算法中获得最佳结果至关重要。

4 Challenges in Offline RL

在本节中, 我们通过分析和评估先前的算法来确定离线强化学习 (RL) 的关键开放性挑战。我们认为这些挑战强调了极简主义方法的重要性, 在这些方法中, 性能可以容易地归因于算法贡献, 而不是与实现细节纠缠在一起。

实现和调参复杂性。 RL 算法实现和调参非常困难 [Henderson et al., 2017, Engstrom et al., 2020, Furuta et al., 2021], 其中轻微的代码级优化和超参数可能对性能和稳定性产生非微不足道的影响。这个问题在离线 RL 的背景下可能会进一步放大, 因为在这个背景下评估实现和超参数的变化违背了离线 RL 的本质, 即明确旨在消除环境交互 [Paine et al., 2020, Yang et al., 2020]。

大多数离线 RL 算法都基于现有的离策略深度 RL 算法构建, 如 TD3 [Fujimoto et al., 2018] 或 SAC [Haarnoja et al., 2018], 但随后通过“非算法”实现变化进一步修改基础算法, 例如修改网络架构、学习率或预训练行为者网络。我们指出, 一个理想的离线 RL 算法应该尽可能少地修改基础算法, 以减少为达到高性能所需的调整空间。

在Table 1中, 我们检查了两种最近的离线 RL 算法——CQL [Kumar et al., 2020] 和 Fisher-BRC [Kostrikov et al., 2021]所做的特定修改。除了算法上的变化外, CQL 还增加了一个预训练阶段, 行为者仅通过模仿学习进行训练, 并在评估期间从策略样本集中选择最大动作。Fisher-BRC 向每个转换添加一个恒定的奖励奖金。这两种方法都通过移除目标更新中的熵项和修

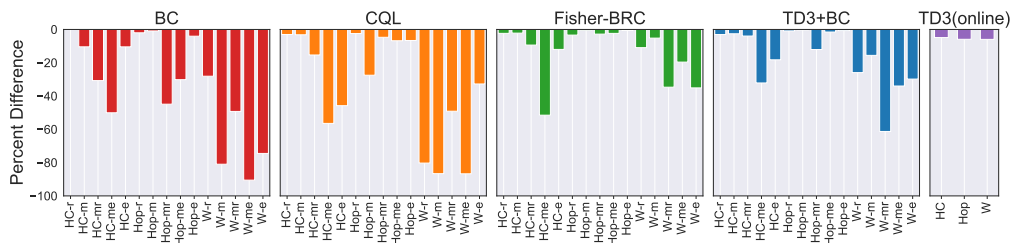


图 2: 在最后评估时10个评估剧集中最差剧集的百分比差异。这衡量了单个时间点上的性能偏差。HC = HalfCheetah, Hop = Hopper, W = Walker, r = 随机, m = 中等, mr = 中等-重放, me = 中等-专家, e = 专家。虽然在线算法（如TD3）通常具有较小的每策略剧集方差（因为它们应在收敛时如此），但所有离线算法对于训练策略的剧集方差令人惊讶地高。

改默认的网络架构来修改 SAC。这些变化增加了额外的超参数，可能需要调优或增加计算成本。

在线环境中，这些变化相对不重要，并且可以通过一些简单的实验进行验证。然而，在离线环境中，由于我们不能与环境互动，对基础算法进行额外调整应被视为更加昂贵，因为验证其有效性的步骤不再是简单地额外增加一步。这也是一个问题，因为与这些论文提出的算法变更不同，这些实现细节缺乏充分的理由，这意味着对于何时包含这些细节或如何通过最少的实验进行调整，几乎没有直观的认识。以 D4RL 基准在 MuJoCo 任务上的情况为例 [Todorov et al., 2012, Fu et al., 2020]，我们有一个强烈的事前假设，即我们的基础深度 RL 算法在此类领域表现良好，因为 SAC/TD3 被认为是这些领域的最先进（或接近最先进）。如果需要额外的变更，那么这表明仅靠算法贡献是不够的。

在Figure 1中，我们考察了在删除 CQL 和 Fisher-BRC 中的实现变更后性能百分比的变化。许多任务的性能大幅下降。这并不一定意味着这些算法的末日，因为当然这些变更在尝试新的数据集和领域时可以保留而无需调优。然而，由于两篇论文都没有提到训练/验证分割，我们只能假设这些变更主要基于它们的评估数据集（在这种情况下是 D4RL），并指出没有足够的证据表明这些变更可能是普遍适用的。最终，我们提出这一点并非为了指出现有算法的根本缺陷，而是建议在进行最小的调整以减少超参数调优需求方面应有所偏好。

额外计算需求。极简主义的另一个动机是避免与修改基础算法（特别是架构）和更复杂的算法思想相关的额外计算成本。在Table 3（包含在Section 6中）中，我们检查了离线 RL 算法的运行时间以及相对于基础算法的成本变化，发现这些修改存在显著的计算成本。除了架构变化外，对于 CQL 来说，这主要是由于多次抽样动作的 $\log\text{sumexp}$ 成本；对于 Fisher-BRC，则是因为独立生成模型的训练成本。由于离线 RL 的一个目标是利用现有的、可能庞大的数据集，因此应该倾向于可扩展且高效的解决方案。当然，运行时间不应该以牺牲性能为代价，但正如我们稍后将展示的那样，存在一种简单且计算自由的方法，可以在当前最先进的算法中匹配性能。

训练策略的不稳定性。在分析先前离线算法最终训练的策略时，我们注意到了旁支且开放的挑战形式——不稳定性。在线强化学习（RL）中，如果当前策略不尽如人意，我们可以使用策略前几次迭代的检查点，或者直接继续训练。然而，在离线RL中，评估应仅发生一次，这大大增加了评估时刻单一策略的重要性。我们在离线RL中突出了两种不稳定性的版本。图Figure 2显示，与在线训练策略收敛到一个稳健的、低方差策略相比，离线训练策略在同一评估期间表现出巨大的性能差异。因此，即使平均性能是合理的，代理在某些回合中的表现仍可能很差。图Figure 3显示了评估集上的不稳定性，这意味着代理的性能可能取决于选择进行评估的具体停止点。这一问题质疑了离线RL对于关键实际应用的有效

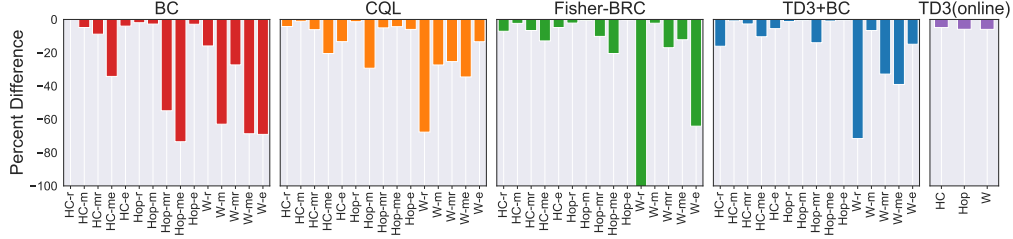


图 3: 最后10次评估中表现最差的百分比差异。这衡量了在一段时间内的性能偏差。HC = HalfCheetah, Hop = Hopper, W = Walker, r = random, m = medium, mr = medium-replay, me = medium-expert, e = expert。类似于 Figure 2 中的结果, 所有离线训练的策略在训练的最后阶段都表现出显著的方差, 而在线设置中则不存在这种方差。

性[Mandel et al., 2014, Gottesman et al., 2018, Gauci et al., 2018, Jaques et al., 2019, Matsushima et al., 2020], 以及目前离线基准测试中仅报告最终策略均值的现状[Fu et al., 2020]。

这些方差不太可能是由高策略熵（例如, TD3+BC 训练了一个确定性策略）引起的, 而是我们的假设认为这个问题是由分布偏移问题和离线训练导致的在未观察状态下的不良泛化能力所致[Ross et al., 2011], 其中优化后的策略从未被允许在环境中执行, 类似于BC的情况。这一离线RL算法的特性在我们评估的所有离线算法中似乎是普遍存在的, 甚至在我们简单的TD3+BC中也是如此, 它与TD3仅有几行代码的差别。尽管我们在这项工作中未能充分解决这一挑战, 但即使在这种简约变体中也可再现的事实证明, 这是一个所有离线训练设置共享的基本问题, 并且是未来社区需要研究的关键问题。

5 A Minimalist Offline RL Algorithm

离线强化学习中的一个关键问题, 外推误差, 可以概括为无法正确评估分布外的动作。因此, 已经出现了多种不同的方法来限制或正则化动作选择, 使得所学策略更容易用给定的数据集进行评估。我们注意到, 虽然最小化例如KL散度是一种合理且有效的方法来减少外推误差, 但没有根本性的论点说明为什么最小化一种散度或距离度量应该比另一种更好。因此, 与其提出一种全新的方法, 我们专注于简洁性, 提出了一个仅需对现有深度强化学习算法进行最少修改的离线RL算法。如Section 4所述, 简约方法具有多种优势, 例如减少需要调整的超参数数量, 通过降低计算成本提高可扩展性, 并提供了一种分析问题的途径, 即通过分离算法贡献与实现细节。

现在我们描述这种离线RL方法。我们的算法基于TD3 [Fujimoto et al., 2018], 仅做了两个简单的改动。首先, 我们在TD3的标准策略更新步骤中添加了一个行为克隆正则化项, 以推动策略偏好数据集 $\mathcal{D}$ 中的动作:

$$\pi = \operatorname{argmax}_{\pi} \mathbb{E}_{s \sim \mathcal{D}} [Q(s, \pi(s))] \rightarrow \pi = \operatorname{argmax}_{\pi} \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\lambda Q(s, \pi(s)) - (\pi(s) - a)^2 \right]. \quad (3)$$

其次, 我们对提供的数据集中每个状态的特征进行归一化。设 s_i 为状态 s 的第 i 个特征, μ_i 和 σ_i 分别为数据集中第 i 个特征的均值和标准差:

$$s_i = \frac{s_i - \mu_i}{\sigma_i + \epsilon}, \quad (4)$$

其中 ϵ 是一个很小的归一化常数（我们使用 10^{-3} ）。虽然我们注意到这是一个在许多深度强化学习算法中常用的实现细节 [Raffin et al., 2019], 但我们强调（1）我们希望对所有实现更改完全透明, 以及（2）归一化在离线强化学习中提供了一个非平凡的性能优势, 特别是在数据集保持固定的情况下。

		BC	BRAC-p	AWAC	CQL	Fisher-BRC	TD3+BC
Random	HalfCheetah	2.0 $\pm$ 0.1	23.5	2.2	21.7 $\pm$ 0.9	32.2 $\pm$ 2.2	10.2 $\pm$ 1.3
	Hopper	9.5 $\pm$ 0.1	11.1	9.6	10.7 $\pm$ 0.1	11.4 $\pm$ 0.2	11.0 $\pm$ 0.1
	Walker2d	1.2 $\pm$ 0.2	0.8	5.1	2.7 $\pm$ 1.2	0.6 $\pm$ 0.6	1.4 $\pm$ 1.6
Medium	HalfCheetah	36.6 $\pm$ 0.6	44.0	37.4	37.2 $\pm$ 0.3	41.3 $\pm$ 0.5	42.8 $\pm$ 0.3
	Hopper	30.0 $\pm$ 0.5	31.2	72.0	44.2 $\pm$ 10.8	99.4 $\pm$ 0.4	99.5 $\pm$ 1.0
	Walker2d	11.4 $\pm$ 6.3	72.7	30.1	57.5 $\pm$ 8.3	79.5 $\pm$ 1.0	79.7 $\pm$ 1.8
Medium Replay	HalfCheetah	34.7 $\pm$ 1.8	45.6	-	41.9 $\pm$ 1.1	43.3 $\pm$ 0.9	43.3 $\pm$ 0.5
	Hopper	19.7 $\pm$ 5.9	0.7	-	28.6 $\pm$ 0.9	35.6 $\pm$ 2.5	31.4 $\pm$ 3.0
	Walker2d	8.3 $\pm$ 1.5	-0.3	-	15.8 $\pm$ 2.6	42.6 $\pm$ 7.0	25.2 $\pm$ 5.1
Medium Expert	HalfCheetah	67.6 $\pm$ 13.2	43.8	36.8	27.1 $\pm$ 3.9	96.1 $\pm$ 9.5	97.9 $\pm$ 4.4
	Hopper	89.6 $\pm$ 27.6	1.1	80.9	111.4 $\pm$ 1.2	90.6 $\pm$ 43.3	112.2 $\pm$ 0.2
	Walker2d	12.0 $\pm$ 5.8	-0.3	42.7	68.1 $\pm$ 13.1	103.6 $\pm$ 4.6	101.1 $\pm$ 9.3
Expert	HalfCheetah	105.2 $\pm$ 1.7	3.8	78.5	82.4 $\pm$ 7.4	106.8 $\pm$ 3.0	105.7 $\pm$ 1.9
	Hopper	111.5 $\pm$ 1.3	6.6	85.2	111.2 $\pm$ 2.1	112.3 $\pm$ 0.2	112.2 $\pm$ 0.2
	Walker2d	56.0 $\pm$ 24.9	-0.2	57.0	103.8 $\pm$ 7.6	79.9 $\pm$ 32.4	105.7 $\pm$ 2.7
Total		595.3 $\pm$ 91.5	284.1	-	764.3 $\pm$ 61.5	974.6 $\pm$ 108.3	979.3 $\pm$ 33.4

表 2: 在最后10次评估和5个种子上的平均归一化得分。最高的表现得分已被突出显示。CQL和Fisher-BRC使用作者提供的实现重新运行，以确保相同的评估过程，而BRAC和AWAC使用先前报告的结果。 $\pm$ 表示各种子的标准差。尽管TD3+BC的实现和调优要简单得多，并且计算成本降低了超过一半，但它实现了与最先进的Fisher-BRC几乎相同的性能。

虽然 Equation (3) 中 λ 的选择最终只是一个超参数，我们观察到 RL（在最大化 Q 方面）和模仿（在最小化 BC 项方面）之间的平衡高度依赖于 Q 的尺度。如果我们假设动作范围为 $[-1, 1]$ ，BC 项最多为 4，然而 Q 的范围将取决于奖励的规模。因此，我们可以在 λ 中加入一个归一化项。给定包含 N 个转移 (s_i, a_i) 的数据集，我们定义标量 λ 为：

$$\lambda = \frac{\alpha}{\frac{1}{N} \sum_{(s_i, a_i)} |Q(s_i, a_i)|}. \quad (5)$$

这是基于 Q 的平均绝对值的一个简单的归一化项。实际上，我们是在小批次上而不是在整个数据集上估计这个均值项。尽管该术语包括 Q ，但不会对其进行微分，而只是用于缩放损失。这种表述的好处是可以在任务之间规范化学习率，因为梯度 $\nabla_a Q(s, a)$ 也会依赖于 Q 的尺度。在我们的实验中，我们使用 $\alpha = 2.5$ 。

这完成了对 TD3+BC 的描述。方程 (3)、(4) 和 (5) 概括了我们对 TD3 所做的所有更改，只需修改大多数代码库中的几行代码即可实现。

6 Experiments

我们在 D4RL 基准上评估了我们提出的方法，该基准包括 OpenAI gym MuJoCo 任务 [Todorov et al., 2012, Brockman et al., 2016, Fu et al., 2020]，涵盖了各种数据集设置和领域。我们的离线强化学习基线包括两种最先进的算法，CQL [Kumar et al., 2020] 和 Fisher-BRC [Kostrikov et al., 2021]，以及 BRAC [Wu et al., 2019] 和 AWAC [Nair et al., 2020]，因为它们的算法简单性。

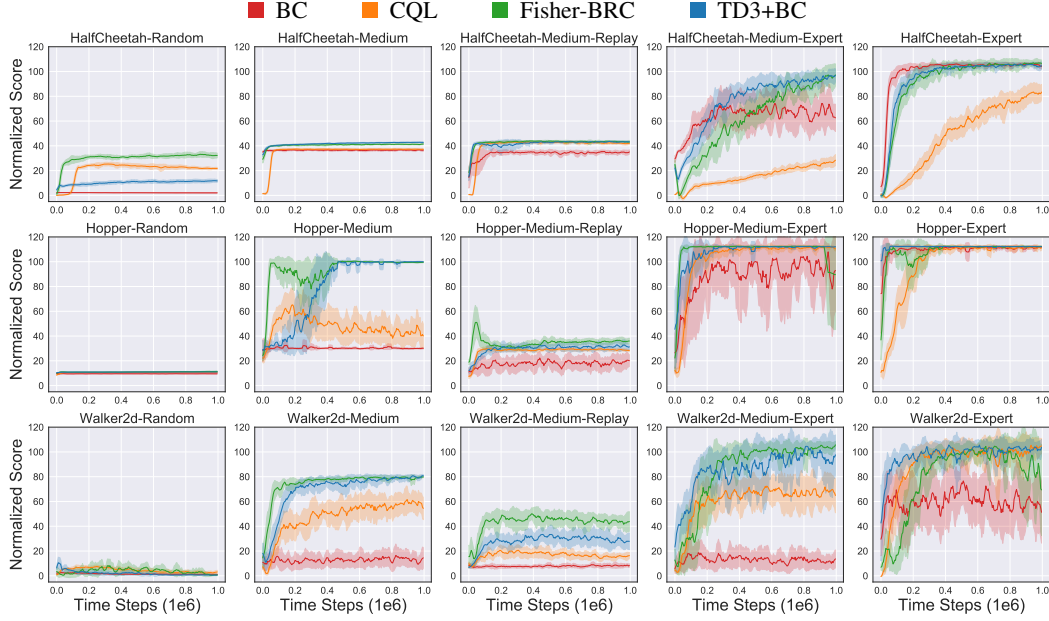


图 4: 学习曲线比较了TD3+BC与D4RL数据集中离线RL基线的性能。曲线是通过对5个种子取平均值得到的, 阴影区域代表了种子间的标准差。TD3+BC展现了与最先进的Fisher-BRC相似的学习速度和最终性能, 而无需预训练生成模型。

	CQL (GitHub)	Fisher-BRC (GitHub)	Fisher-BRC (Ours)	TD3+BC (Ours)
Implementation	25m	39m	15m	< 1s
Algorithmic	1h 29m	33m	58m	< 5s
Total	4h 11m	2h 12m	2h 8m	39m

Run Time

表 3: 各离线强化学习算法的运行时间比较 (不包括评估成本)。(左) 每种方法所增加的实现细节 (主要是架构变化) 和算法细节的成本分解。(右) 每个算法的总训练时间。尽管CQL和Fisher-BRC由于各种增加的复杂性 (例如参见Table 1) 而比其基础在线强化学习算法显著增加了计算成本, TD3+BC则几乎没有增加。这导致其计算成本不到这些先前最先进算法的一半。

为了确保算法之间公平且一致的实验评估, 我们使用作者提供的实现²³重新运行最先进的算法CQL和Fisher-BRC。我们对每个算法训练1百万个时间步, 并每5000个时间步进行一次评估。每次评估包括10个回合。BRAC的结果来自D4RL基准测试和CQL论文, AWAC的结果直接从论文中获得。BC结果使用我们自己的实现获得。

D4RL. 我们在Table 2中报告最终性能结果, 并在Figure 4中显示学习曲线。尽管我们的方法非常简单, 但在大多数任务中, 它超越或匹配了当前最先进的离线强化学习算法的性能。只有Fisher-BRC表现出相当的性能⁴。通过检查学习曲线, 我们可以看到, 我们的方法实现了类似的學習速度和稳定性, 而无需任何预训练阶段。

²<https://github.com/aviralkumar2907/CQL>

³https://github.com/google-research/google-research/tree/master/fisher_brc

⁴如先前论文 [Kostrikov et al., 2021]所述, 我们注意到作者提供的CQL实现比他们报告的结果要低。

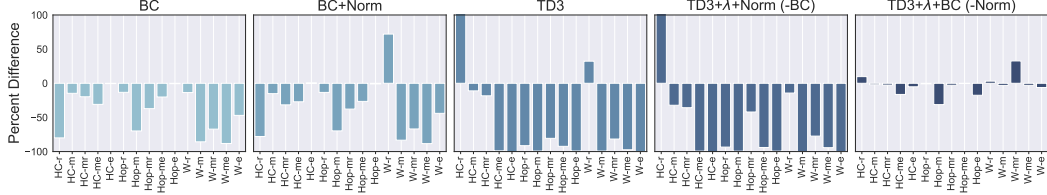


图 5: 我们提出的方法的消融实验与完整算法性能的百分比差异。TD3+λ+BC+Norm 指的是完整算法, 其中 Norm 表示状态特征归一化。HC = HalfCheetah, Hop = Hopper, W = Walker, r = 随机, m = 中等, mr = 中等 Replay, me = 中等专家, e = 专家。正如预期的那样, BC 和 TD3 都是实现高性能的必要组成部分。虽然移除状态归一化对算法性能的影响不大, 但我们注意到它在许多任务上都能提供性能提升, 同时是一个简单的添加。

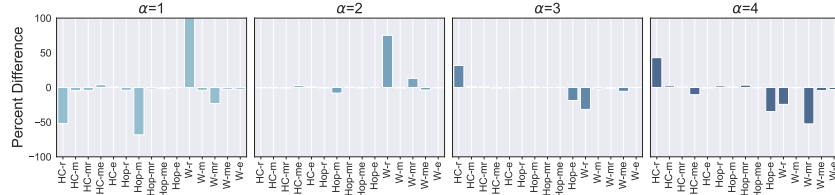


图 6: 在 α 上的消融与完整算法相比的性能百分比差异。回想唯一的超参数形式为 $\lambda = \frac{\alpha}{\sum |Q(s,a)|}$, 其中 λ 在策略更新时对 Q 函数进行加权。默认使用 $\alpha = 2.5$ 。HC = HalfCheetah, Hop = Hopper, W = Walker, r = 随机, m = 中等, mr = 中等-回放, me = 中等-专家, e = 专家。虽然 $\alpha = 2$ 和 $\alpha = 3$ 之间的性能几乎没有差异, 但我们注意到, 当算法开始更接近强化学习 ($\alpha = 4$) 或模仿学习 ($\alpha = 1$) 时, 选择任务的性能开始下降。

运行时间. 我们评估每种离线强化学习算法训练1百万个时间步的运行时间, 使用作者提供的实现。此外, 为了公平比较, 我们重新实现了Fisher-BRC, 使得每种方法都在同一框架 (PyTorch [Paszke et al., 2019]) 中进行比较。结果见Table 3。不出所料, 我们发现我们的方法在实际训练时间上比以前的方法更具有优势, 对底层TD3算法几乎没有增加成本。所有运行时间实验均在单个GeForce GTX 1080 GPU和频率为4.00GHz的Intel Core i7-6700K CPU上运行。

消融研究. 在Figure 5中, 我们对我们的方法的各个组件进行了消融研究。如先前工作 [Fujimoto et al., 2019b]所述, 如果没有行为克隆正则化, 仅凭RL算法本身不足以实现高性能 (除了在某些随机数据集上)。我们还注意到, 即使在专家任务上, 我们的算法也不会逊色于传统的行为克隆。可以预见的是, 状态归一化的移除对性能的影响最小, 但仍然在一系列任务中提供了一些好处, 而且是一个较小的调整。在Figure 6中, 我们评估了算法对超参数 α 的敏感性, 其中价值函数上的权重 $\lambda = \frac{\alpha}{\sum_{(s,a)} |Q(s,a)|}$ 由 α 决定。在许多任务中, 我们的方法对此权重因子具有鲁棒性, 但需要注意的是, 在部分任务中, 随着 α 开始更重地偏向模仿 ($\alpha = 1$) 或强化学习 ($\alpha = 4$), 性能开始下降。

7 Conclusion

最近在离线强化学习 (RL) 中的大多数进展都集中在通过对策略动作进行正则化以使其靠近批量数据的支持范围内的想法上, 然而, 许多最先进的算法除了基础算法之外还有显著的复杂性和额外的修改, 这不仅导致运行时间更慢, 还使得性能提升的来源难以追踪。我们并不追求复杂性而是优化简洁性, 并介绍了一个最小化的算法, 该算法仅需对基础TD3算

法做出几行更改就能实现最先进的性能，计算时间不到竞争算法的一半，并且只有一个额外的超参数。

此外，我们强调了离线RL研究中现有的开放挑战，包括我们在本工作中成功解决的额外实施、计算和超参数调优的复杂性，但也指出社区应该在未来的研究和基准测试中关注的一个被忽视的问题：与在线训练相比，离线训练策略的高剧集方差（见图 2 和 3）。最后，我们认为我们的方法的纯粹简单性突显了社区可能对算法复杂性的过度重视，我们希望激励未来的工作重新审视可能被忽略的更简单的替代方案。

参考文献

- Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*, 2016.
- Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Leonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *International Conference on Learning Representations*, 2021.
- Christopher G Atkeson and Stefan Schaal. Robot learning from demonstration. In *Proceedings of the Fourteenth International Conference on Machine Learning*, pages 12–20, 1997.
- Jonathan Booyer. Bc+rl: Imitation learning from non-optimal demonstrations. URL <https://web.stanford.edu/~jaustinb/papers/CS234.pdf>.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.
- Jacob Buckman, Carles Gelada, and Marc G Bellemare. The importance of pessimism in fixed-dataset policy optimization. *arXiv preprint arXiv:2009.06799*, 2020.
- Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *arXiv preprint arXiv:2106.01345*, 2021.
- Xinyue Chen, Zijian Zhou, Zheng Wang, Che Wang, Yanqiu Wu, and Keith Ross. Bail: Best-action imitation learning for batch deep reinforcement learning. *Advances in Neural Information Processing Systems*, 33, 2020.
- Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep rl: A case study on ppo and trpo. In *International Conference on Learning Representations*, 2020.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. 2020.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, volume 80, pages 1587–1596. PMLR, 2018.
- Scott Fujimoto, Edoardo Conti, Mohammad Ghavamzadeh, and Joelle Pineau. Benchmarking batch deep reinforcement learning algorithms. *arXiv preprint arXiv:1910.01708*, 2019a.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, pages 2052–2062, 2019b.
- Hiroki Furuta, Tadashi Kozuno, Tatsuya Matsushima, Yutaka Matsuo, and Shixiang Shane Gu. Identifying co-adaptation of algorithmic and implementational innovations in deep reinforcement learning: A taxonomy and case study of inference-based algorithms. *arXiv preprint arXiv:2103.17258*, 2021.

- Jason Gauci, Edoardo Conti, Yitao Liang, Kittipat Virochsiri, Yuchen He, Zachary Kaden, Vivek Narayanan, Xiaohui Ye, Zhengxing Chen, and Scott Fujimoto. Horizon: Facebook’s open source applied reinforcement learning platform. *arXiv preprint arXiv:1811.00260*, 2018.
- Seyed Kamyar Seyed Ghasemipour, Dale Schuurmans, and Shixiang Shane Gu. Emaq: Expected-max q-learning operator for simple yet effective offline and online rl. *arXiv preprint arXiv:2007.11091*, 2020.
- Vinicius G Goecks, Gregory M Gremillion, Vernon J Lawhern, John Valasek, and Nicholas R Waytowich. Integrating behavior cloning and reinforcement learning for improved performance in dense and sparse reward environments. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, pages 465–473, 2020.
- Omer Gottesman, Fredrik Johansson, Joshua Meier, Jack Dent, Donghun Lee, Srivatsan Srinivasan, Linying Zhang, Yi Ding, David Wihl, Xuefeng Peng, et al. Evaluating reinforcement learning algorithms in observational health settings. *arXiv preprint arXiv:1805.12298*, 2018.
- Caglar Gulcehre, Tom Le Paine, Bobak Shahriari, Misha Denil, Matt Hoffman, Hubert Soyer, Richard Tanburn, Steven Kapturowski, Neil Rabinowitz, Duncan Williams, Gabriel Barth-Maron, Ziyu Wang, Nando de Freitas, and Worlds Team. Making efficient use of demonstrations to solve hard exploration problems. In *International Conference on Learning Representations*, 2020.
- Yijie Guo, Shengyu Feng, Nicolas Le Roux, Ed Chi, Honglak Lee, and Minmin Chen. Batch reinforcement learning through continuation method. In *International Conference on Learning Representations*, 2021.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, volume 80, pages 1861–1870. PMLR, 2018.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2017.
- Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, et al. Deep q-learning from demonstrations. *arXiv preprint arXiv:1704.03732*, 2017.
- Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. *arXiv preprint arXiv:1907.00456*, 2019.
- Kshitij Judah, Alan Fern, Prasad Tadepalli, and Robby Goetschalckx. Imitation learning with demonstrations and shaping rewards. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 28, 2014.
- Sham Kakade. A natural policy gradient. In *Proceedings of the 14th International Conference on Neural Information Processing Systems: Natural and Synthetic*, pages 1531–1538, 2001.
- Bingyi Kang, Zequn Jie, and Jiashi Feng. Policy optimization with demonstrations. In *International Conference on Machine Learning*, pages 2469–2478. PMLR, 2018.

- Beomjoon Kim, Amir-massoud Farahmand, Joelle Pineau, and Doina Precup. Learning from limited demonstrations. In *Advances in Neural Information Processing Systems*, pages 2859–2867, 2013.
- Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Ilya Kostrikov, Jonathan Tompson, Rob Fergus, and Ofir Nachum. Offline reinforcement learning with fisher divergence critic regularization. *arXiv preprint arXiv:2103.08050*, 2021.
- Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. In *Advances in Neural Information Processing Systems*, pages 11784–11794, 2019.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *arXiv preprint arXiv:2006.04779*, 2020.
- Sascha Lange, Thomas Gabel, and Martin Riedmiller. Batch reinforcement learning. In *Reinforcement learning*, pages 45–73. Springer, 2012.
- Romain Laroché, Paul Trichelair, and Remi Tachet Des Combes. Safe policy improvement with baseline bootstrapping. In *International Conference on Machine Learning*, pages 3652–3661. PMLR, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Travis Mandel, Yun-En Liu, Sergey Levine, Emma Brunskill, and Zoran Popovic. Offline policy evaluation across representations with applications to educational games. In *International Conference on Autonomous Agents and Multiagent Systems*, 2014.
- Tatsuya Matsushima, Hiroki Furuta, Yutaka Matsuo, Ofir Nachum, and Shixiang Gu. Deployment-efficient reinforcement learning via model-based offline optimization. *arXiv preprint arXiv:2006.03647*, 2020.
- Ofir Nachum, Bo Dai, Ilya Kostrikov, Yinlam Chow, Lihong Li, and Dale Schuurmans. Algaedice: Policy gradient from arbitrary experience. *arXiv preprint arXiv:1912.02074*, 2019.
- Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6292–6299. IEEE, 2018.
- Ashvin Nair, Murtaza Dalal, Abhishek Gupta, and Sergey Levine. Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- Ian Osband, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva, Katrina McKinney, Tor Lattimore, Csaba Szepesvari, Satinder Singh, et al. Behaviour suite for reinforcement learning. *arXiv preprint arXiv:1908.03568*, 2019.

- Tom Le Paine, Cosmin Paduraru, Andrea Michi, Caglar Gulcehre, Konrad Zolna, Alexander Novikov, Ziyu Wang, and Nando de Freitas. Hyperparameter selection for offline reinforcement learning. *arXiv preprint arXiv:2007.09055*, 2020.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pages 8024–8035, 2019.
- Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- Mark Pfeiffer, Samarth Shukla, Matteo Turchetta, Cesar Cadena, Andreas Krause, Roland Siegwart, and Juan Nieto. Reinforced imitation: Sample efficient deep reinforcement learning for mapless navigation by leveraging prior demonstrations. *IEEE Robotics and Automation Letters*, 3(4): 4423–4430, 2018.
- Dean A Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural computation*, 3(1):88–97, 1991.
- Antonin Raffin, Ashley Hill, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, and Noah Dormann. Stable baselines3. <https://github.com/DLR-RM/stable-baselines3>, 2019.
- Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.
- Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Noah Siegel, Jost Tobias Springenberg, Felix Berkenkamp, Abbas Abdolmaleki, Michael Neunert, Thomas Lampe, Roland Hafner, Nicolas Heess, and Martin Riedmiller. Keep doing what worked: Behavior modelling priors for offline reinforcement learning. In *International Conference on Learning Representations*, 2020.
- David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International Conference on Machine Learning*, pages 387–395, 2014.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012.

- George Tucker, Surya Bhupatiraju, Shixiang Gu, Richard Turner, Zoubin Ghahramani, and Sergey Levine. The mirage of action-dependent baselines in reinforcement learning. In *International Conference on Machine Learning*, 2018.
- Matej Večerík, Todd Hester, Jonathan Scholz, Fumin Wang, Olivier Pietquin, Bilal Piot, Nicolas Heess, Thomas Rothörl, Thomas Lampe, and Martin Riedmiller. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*, 2017.
- Qing Wang, Jiechao Xiong, Lei Han, Peng Sun, Han Liu, and Tong Zhang. Exponentially weighted imitation learning for batched historical data. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, pages 6291–6300, 2018.
- Ziyu Wang, Alexander Novikov, Konrad Zolna, Josh S Merel, Jost Tobias Springenberg, Scott E Reed, Bobak Shahriari, Noah Siegel, Caglar Gulcehre, Nicolas Heess, et al. Critic regularized regression. *Advances in Neural Information Processing Systems*, 33, 2020.
- Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361*, 2019.
- Yuchen Wu, Melissa Mozifian, and Florian Shkurti. Shaping rewards for reinforcement learning with imperfect demonstrations using generative models. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6628–6634. IEEE, 2021.
- Mengjiao Yang, Bo Dai, Ofir Nachum, George Tucker, and Dale Schuurmans. Offline policy selection under uncertainty. *arXiv preprint arXiv:2012.06919*, 2020.
- Yuke Zhu, Ziyu Wang, Josh Merel, Andrei Rusu, Tom Erez, Serkan Cabi, Saran Tunyasuvunakool, János Kramár, Raia Hadsell, Nando de Freitas, et al. Reinforcement and imitation learning for diverse visuomotor skills. *arXiv preprint arXiv:1802.09564*, 2018.

A Broader Impact

离线强化学习 (RL) 将通过利用现有数据集 (如机器人技术或医疗保健应用中的离线记录数据) 启用新的强化学习应用, 对社会产生影响。在这些领域中, 数据收集困难、时间受限、成本高昂等。这可能包括一些潜在的负面影响, 例如在社交媒体上促进成瘾行为。离线RL的另一个限制是, 它受到数据集中任何偏差的影响, 并且可以被生成数据的策略所影响。

对于我们的具体算法TD3+BC, 鉴于其相对于现有最先进方法的性能提升有限, 看到我们在这些背景下取得重大影响将会令人惊讶。然而, 我们可能会在降低计算成本或减少实现负担方面产生影响, 从而使新用户更容易访问离线RL。换句话说, 我们认为我们工作的影响力在于可访问性和易用性, 而不是通过改变可能应用的范围。

B Experimental Details

软件。我们使用以下软件版本:

- Python 3.6
- Pytorch 1.4.0 [Paszke et al., 2019]
- Tensorflow 2.4.0 [Abadi et al., 2016]
- Gym 0.17.0 [Brockman et al., 2016]
- MuJoCo 1.50⁵ [Todorov et al., 2012]
- mujoco-py 1.50.1.1

所有 D4RL 数据集 [Fu et al., 2020] 使用 v0 版本。

超参数。我们对 TD3⁶ [Fujimoto et al., 2018]、CQL⁷ [Kumar et al., 2020] 和 Fisher-BRC⁸ [Kostrikov et al., 2021] 的实现都是基于各自的作者在 GitHub 上提供的实现。对于 TD3+BC, 仅在 Hopper-medium-v0 和 Hopper-expert-v0 上使用单个种子在 (1, 2, 2.5, 3, 4) 范围内调整了 α , 此种子未用于最终报告的结果中。我们在可能的情况下根据每个 GitHub 使用默认的超参数。对于 CQL, 我们修改了 GitHub 默认的演员学习率, 并使用固定的 α 而不是拉格朗日变体, 以匹配他们在论文中定义的超参数 (这与 GitHub 中的不同), 因为我们的实验结果表明原始的超参数效果更好。我们重新实现了 Fisher-BRC (使用 PyTorch 而非 Tensorflow) 仅用于运行时实验。

我们在 Table 4 中概述了 TD3+BC 使用的超参数, 在 Table 5 中概述了 CQL 使用的超参数, 以及在 Table 6 中概述了 Fisher-BRC 使用的超参数。

选择 λ 的启发式方法。虽然我们发现单一设置的 λ 可以适用于所有数据集, 但一些从业者可能对选择 λ 的指导原则感兴趣, 例如将其设为固定常数。请注意, 我们的归一化启发式的目的是为了平衡价值最大化和行为克隆的重要性。通过领域知识, 可以绕过这一启发式方法, 选择大约等于 α 的 λ 。我们还可以通过考虑代理的价值估计来选择 λ ——如果我们看到由于外推误差 [Fujimoto et al., 2019b] 导致的价值函数发散, 则需要降低 λ 使得 BC 项具有更高的权重。或者, 如果性能类似于行为代理的性能, 则应考虑更高的 λ 值。

⁵许可信息:<https://www.roboti.us/license.html>

⁶<https://github.com/sfujim/TD3,commit/6a9f76101058d674518018ffbb532f5a652c1d37>

⁷<https://github.com/aviralkumar2907/CQL,commit/d67dbe9cf5d2b96e3b462b6146f249b3d6569796>

⁸https://github.com/google-research/google-research/tree/master/fisher_brc,commit/9898bb462a79e727ca5f413dba0ac3c4ee48d6c0

	Hyperparameter	Value
TD3 Hyperparameters	Optimizer	Adam [Kingma and Ba, 2014]
	Critic learning rate	3e-4
	Actor learning rate	3e-4
	Mini-batch size	256
	Discount factor	0.99
	Target update rate	5e-3
	Policy noise	0.2
	Policy noise clipping	(-0.5, 0.5)
	Policy update frequency	2
Architecture	Critic hidden dim	256
	Critic hidden layers	2
	Critic activation function	ReLU
	Actor hidden dim	256
	Actor hidden layers	2
	Actor activation function	ReLU
TD3+BC Hyperparameters	α	2.5

表 4: TD3+BC 超参数。回顾 $\lambda = \frac{\alpha}{\frac{1}{N} \sum_{(s,a)} |Q(s,a)|}$ 的形式。TD3 的超参数未从 TD3 GitHub 进行修改。

	Hyperparameter	Value
SAC Hyperparameters	Optimizer	Adam [Kingma and Ba, 2014]
	Critic learning rate	3e-4
	Actor learning rate	3e-5 [†]
	Mini-batch size	256
	Discount factor	0.99
	Target update rate	5e-3
	Target entropy	-1 · Action Dim
	Entropy in Q target	False [†]
Architecture	Critic hidden dim	256
	Critic hidden layers	3 [†]
	Critic activation function	ReLU
	Actor hidden dim	256
	Actor hidden layers	3 [†]
	Actor activation function	ReLU
CQL Hyperparameters	Lagrange	False
	α	10
	Pre-training steps	40e3
	Num sampled actions (during eval)	10
	Num sampled actions (logsumexp)	10

表 5: CQL 超参数。我们使用了 CQL 论文中定义的超参数，而不是 CQL GitHub 中的默认设置，因为我们发现这些设置表现不佳。[†] 表示与原始 SAC 超参数偏离的超参数。

	Hyperparameter	Value
SAC Hyperparameters	Optimizer	Adam [Kingma and Ba, 2014]
	Critic Learning Rate	3e-4
	Actor Learning Rate	3e-4
	Mini-batch size	256
	Discount factor	0.99
	Target update rate	5e-3
	Target entropy	-1 · Action Dim
	Entropy in Q target	False [†]
Architecture	Critic hidden dim	256
	Critic hidden layers	3 [†]
	Critic activation function	ReLU
	Actor hidden dim	256
	Actor hidden layers	3 [†]
	Actor activation function	ReLU
Generative Model Hyperparameters	Num Gaussians	5
	Optimizer	Adam [Kingma and Ba, 2014]
	Learning rate	(1e-3, 1e-4, 1e-5)
	Learning rate schedule	Piecewise linear (0, 8e5, 9e5)
	Target entropy	-1 · Action Dim
Generative Model Architecture	Hidden dim	256
	Hidden layers	2
	Activation function	ReLU
Fisher-BRC Hyperparameters	Gradient penalty λ	0.1
	Reward bonus	5

表 6: Fisher-BRC 超参数。我们使用 Fisher-BRC GitHub 中的默认超参数。[†] 表示与原始 SAC 超参数不同的超参数。

C Additional Experiments

C.1 Additional Datasets

TD3+BC 的一个问题是其在随机数据上的表现较差。在 Table 7 中，我们混合了来自 D4RL [Fu et al., 2020] 的随机和专家数据集，通过从每个数据集中随机选择一半的转换并连接起来。我们发现 TD3+BC 的表现与 Fisher-BRC 相当。然而，两种算法在 Walker2d 上的表现都不如 CQL。这种性能差距的一个假设是因为 BC 在 Walker2d 专家数据集上的表现较差（见正文中的表 2 和图 4）。为了完整性，我们还报告了 TD3+BC 在 D4RL AntMaze 数据集上的表现。对于这个领域，我们发现状态特征归一化对性能有害，因此未包括在内。所有其他超参数保持不变。

	CQL	Fisher-BRC	TD3+BC
HalfCheetah	73.3 $\pm$ 6.9	105.8 $\pm$ 2.5	101.9 $\pm$ 7.6
Hopper	110.8 $\pm$ 2.4	111.9 $\pm$ 0.9	112.2 $\pm$ 0.3
Walker2d	100.3 $\pm$ 8.5	32.0 $\pm$ 37.3	28.8 $\pm$ 23.4

表 7: 在包含50%的随机D4RL数据集和50%的专家D4RL数据集的混合数据集上，最后10次评估和5个种子的平均归一化得分。CQL和Fisher-BRC使用作者提供的实现重新运行，以确保相同的评估过程。 $\pm$ 表示种子间的标准偏差。

	TD3+BC
AntMaze-Umaze	78.6 $\pm$ 33.3
AntMaze-Umaze-Diverse	71.4 $\pm$ 20.7
AntMaze-Medium-Diverse	10.6 $\pm$ 10.1
AntMaze-Medium-Play	3.0 $\pm$ 4.8
AntMaze-Large-Diverse	0.2 $\pm$ 0.4
AntMaze-Large-Play	0.0 $\pm$ 0.0

表 8: 在AntMaze环境下的最后10次评估和5个种子的平均归一化得分。 $\pm$ 表示种子间的标准差。

C.2 State Feature Normalization with Other Algorithms

为了更好地理解状态特征归一化的有效性，我们将该技术应用于CQL和Fisher-BRC。我们在Figure 7中报告了包含此技术的性能百分比差异。我们发现这通常对性能的影响很小。

C.3 Benchmarking against the Decision Transformer

决策变压器（DT）是一项并发工作，该研究通过将问题框架为序列建模来考察在离线强化学习中使用变压器架构的方法 [Chen et al., 2021]。作者使用了D4RL-v2数据集，这非凡地影响了性能，并使得与-v0数据集上的结果进行直接比较变得不准确。为了与这种方法进行比较，我们重新运行了TD3+BC在-v2数据集上。结果报告于Table 9。尽管DT使用了来自语言建模的最先进技术和一些特定环境的超参数，我们发现TD3+BC在没有任何额外超参数调整的情况下达到了类似的性能。此外，我们在Figure 8中使用作者提供的实现⁹对DT和TD3+BC的训练时间进行了基准测试，使用相同的实验设置：一个GeForce GTX

⁹<https://github.com/kzl/decision-transformer>, commit 5fc73c19f1a89cb17e83aa656b6bba1986e9da59

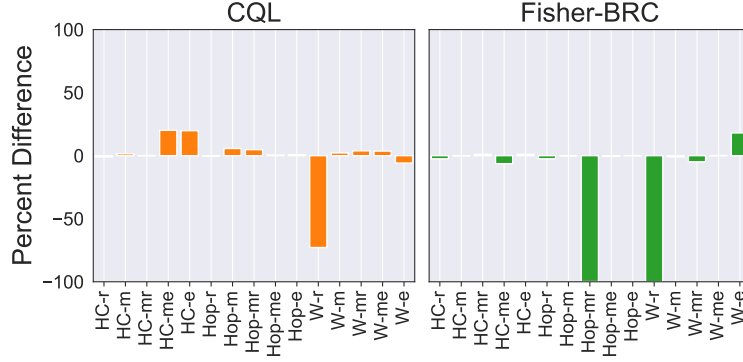


图 7: 离线RL算法在添加状态特征归一化后的性能百分比差异。HC = HalfCheetah, Hop = Hopper, W = Walker, r = 随机, m = 中等, mr = 中等重放, me = 中等专家, e = 专家。

1080 GPU和一个4.00GHz的Intel Core i7-6700K CPU。不出所料，TD3+BC的训练速度显著更快，因为它不依赖于昂贵的变压器架构。

Dataset	Environment	DT	TD3+BC
Random	HalfCheetah	-	11.0 $\pm$ 1.1
	Hopper	-	8.5 $\pm$ 0.6
	Walker2d	-	1.6 $\pm$ 1.7
Medium	HalfCheetah	42.6 $\pm$ 0.1	48.3 $\pm$ 0.3
	Hopper	67.6 $\pm$ 1	59.3 $\pm$ 4.2
	Walker2d	74.0 $\pm$ 1.4	83.7 $\pm$ 2.1
Medium-Replay	HalfCheetah	36.6 $\pm$ 0.8	44.6 $\pm$ 0.5
	Hopper	82.7 $\pm$ 7	60.9 $\pm$ 18.8
	Walker2d	66.6 $\pm$ 3	81.8 $\pm$ 5.5
Medium-Expert	HalfCheetah	86.8 $\pm$ 1.3	90.7 $\pm$ 4.3
	Hopper	107.6 $\pm$ 1.8	98.0 $\pm$ 9.4
	Walker2d	108.1 $\pm$ 0.2	110.1 $\pm$ 0.5
Expert	HalfCheetah	-	96.7 $\pm$ 1.1
	Hopper	-	107.8 $\pm$ 7
	Walker2d	-	110.2 $\pm$ 0.3
Total (DT)		672.6 $\pm$ 16.6	677.4 $\pm$ 45.6
Total		-	1013.2 $\pm$ 57.4

表 9: 使用 D4RL -v2 数据集的平均归一化得分。最高得分被突出显示。 $\pm$ 表示种子的标准差。总分 (DT) 是 DT 结果子任务得分的总和。TD3+BC 结果遵循与 -v0 数据集相同的实验程序，取最后 10 次评估和 5 个种子的平均值。没有进行额外的超参数调整。DT 结果直接取自论文，使用了 3 个种子。TD3+BC 和 DT 达到了可比较的性能。

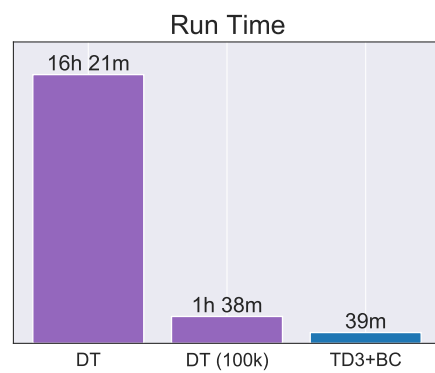


图 8: 基准测试DT和TD3+BC在1百万步训练时间上的实际运行时间。不包括评估成本。我们注意到, DT仅用10万时间步进行训练, 这大大减少了计算成本, 但即使有这种减少, DT的训练时间仍比TD3+BC多出一倍以上。对于许多D4RL任务, TD3+BC的性能在10万时间步之前就已经收敛(见Figure 4中的学习曲线), 但与DT不同的是, 我们可以让TD3+BC运行完整的1百万步而不产生显著的计算成本。